Clase Práctica 4 de Ingeniería de Software

Mani Molido 2, ahora más molido

Integrantes

#	Nombre completo
1	Adrián Xavier Brioso Jurado
2	Alejandro García Ruiz
3	Ana Laura Hernández Cuétino
4	Juan Carlos Yern Espinosa
5	Laura Alonso Rivero

Contribución individual por ejercicio

Integrante	Ejercicio(s) en los que trabajó
Adrián Xavier Brioso Jurado	6a, 6b
Alejandro García Ruiz	6c, 6d
Ana Laura Hernández Cuétino	5a, 5c
Juan Carlos Yern Espinosa	5b
Laura Alonso Rivero	5d

Repositorio público del proyecto

`https://github.com/Jcyern/Management-for-the-automatic-generation-of-delivery-routes/tree/Entregas-IS`

23 de octubre de 2026

Ejercicio 5 — Sistema de Gestión de Archivos Pesados

a)

El patrón que mejor encaja es el Proxy, más precisamente su variante Virtual Proxy, es un sustituto que implemente la misma interfaz que el objeto real y crea/carga el objeto pesado solo en el primer acceso. El cliente sigue trabajando contra `IArchivoPesado` y no sabe si detrás hay un archivo ya cargado o un proxy que lo cargará bajo demanda.

Clases propuestas:

- `IArchivoPesado`: interfaz común con operaciones como `Visualizar()`, `Procesar()`, `Nombre`, `TamanoBytes`.
- `VideoHD`, `DatasetCientifico`, `ImagenSatelital`: objetos reales pesados. Su constructor o su método de carga reserva/lee el contenido.
- `ProxyArchivoPesado`: implementa `IArchivoPesado`, mantiene una referencia perezosa al objeto real y un `Func<IArchivoPesado>` para crearlo.
- `MetricasProxy`: registra accesos, cantidad de cargas y tiempo de carga sin cambiar la interfaz.
- Cliente: usa `IArchivoPesado`; no conoce las clases concretas ni cuándo se cargan.

Una lazy initialization simple dentro del objeto real o del cliente no evita necesariamente crear el objeto real, obliga al cliente a conocer la clase concreta o a repetir lógica de carga y tampoco centraliza métricas ni control de concurrencia. El Virtual Proxy es transparente: el cliente ve la misma interfaz y el proxy decide cuándo instanciar el objeto pesado. Un Protection Proxy controla permisos o derechos de acceso; no está pensado para diferir la creación ni para ahorrar memoria. Podría combinarse con el Virtual Proxy si además se necesitara seguridad, pero no resuelve el problema principal.

b)

```
using System;
using System.Diagnostics;
using System.Threading;

public interface IArchivoPesado
{
    string Nombre { get; }
    long TamanoBytes { get; }
    void Visualizar();
    void Procesar();
}

public sealed class MetricasProxy
{
    private long _accesos;
    private long _cargas;
    private long _msCargaTotal;

    public void RegistrarAcceso() => Interlocked.Increment(ref _accesos);

    public void RegistrarCarga(long ms)
    {
        Interlocked.Increment(ref _cargas);
```

```
        Interlocked.Add(ref _msCargaTotal, ms);
    }

    public long Accesos => Interlocked.Read(ref _accesos);
    public long Cargas => Interlocked.Read(ref _cargas);
    public long MsCargaTotal => Interlocked.Read(ref _msCargaTotal);

    public override string ToString() =>
        $"Accesos={Accesos}, Cargas={Cargas}, TiempoCargaTotal={MsCargaTotal} ms";
}

public sealed class ProxyArchivoPesado : IArchivoPesado
{
    private readonly Func<IArchivoPesado> _cargador;
    private readonly object _candado = new();
    private volatile IArchivoPesado? _real;
    private readonly MetricasProxy _metricas = new();

    public string Nombre { get; }
    public long TamanoBytes { get; }

    public ProxyArchivoPesado(string nombre, long tamanoBytes, Func<IArchivoPesado>
    cargador)
    {
        Nombre = nombre;
        TamanoBytes = tamanoBytes;
        _cargador = cargador;
    }

    public MetricasProxy Metricas => _metricas;

    private IArchivoPesado ObtenerReal()
    {
        var real = _real;
        if (real != null)
        {
            _metricas.RegistrarAcceso();
            return real;
        }

        lock (_candado)
        {
            if (_real == null)
            {
                var sw = Stopwatch.StartNew();
                _real = _cargador();
                sw.Stop();
                _metricas.RegistrarCarga(sw.ElapsedMilliseconds);
            }

            _metricas.RegistrarAcceso();
            return _real;
        }
    }

    public void Visualizar() => ObtenerReal().Visualizar();
    public void Procesar() => ObtenerReal().Procesar();
}
```

```
public sealed class VideoHD : IArchivoPesado
{
    private byte[] _contenido;

    public string Nombre { get; }
    public long TamanoBytes { get; }

    public VideoHD(string nombre, int tamanoBytes)
    {
        Nombre = nombre;
        TamanoBytes = tamanoBytes;
        Thread.Sleep(300);
        _contenido = new byte[tamanoBytes];
    }

    public void Visualizar() => Console.WriteLine($"Reproduciendo video HD:
    {Nombre}");
    public void Procesar() => Console.WriteLine($"Procesando video HD: {Nombre}");
}

public sealed class DatasetCientifico : IArchivoPesado
{
    private byte[] _contenido;

    public string Nombre { get; }
    public long TamanoBytes { get; }

    public DatasetCientifico(string nombre, int tamanoBytes)
    {
        Nombre = nombre;
        TamanoBytes = tamanoBytes;
        Thread.Sleep(400);
        _contenido = new byte[tamanoBytes];
    }

    public void Visualizar() => Console.WriteLine($"Mostrando dataset: {Nombre}");
    public void Procesar() => Console.WriteLine($"Analizando dataset: {Nombre}");
}

public sealed class ImagenSatelital : IArchivoPesado
{
    private byte[] _contenido;

    public string Nombre { get; }
    public long TamanoBytes { get; }

    public ImagenSatelital(string nombre, int tamanoBytes)
    {
        Nombre = nombre;
        TamanoBytes = tamanoBytes;
        Thread.Sleep(250);
        _contenido = new byte[tamanoBytes];
    }

    public void Visualizar() => Console.WriteLine($"Visualizando imagen satelital:
    {Nombre}");
```

```
    public void Procesar() => Console.WriteLine($"Procesando imagen satelital:
    {Nombre}");
}
```

Las métricas se registran dentro del proxy; `IArchivoPesado` no cambia.

c)

Si dos hilos llaman `Visualizar()` sobre el mismo proxy y el objeto real aún no está cargado, ambos podrían intentar crearlo. Para evitarlo se usa **doble comprobación con bloqueo**:

1. Primer chequeo fuera del `lock`: si `_real` ya existe, se retorna rápido.
2. Si no existe, se entra al `lock`.
3. Segundo chequeo dentro del `lock`: solo el primer hilo carga; el segundo espera y luego encuentra `_real` ya asignado.
4. `_real` se declara `volatile` para que la asignación sea visible entre hilos.

En Java el equivalente sería `volatile` + bloque `synchronized`; en C# también puede usarse `Lazy<T>` con `LazyThreadSafetyMode.ExecutionAndPublication`. Las métricas usan `Interlocked` para incrementos atómicos. Así se evita carga duplicada y se mantiene la interfaz sin cambios.

d)

Sin proxy, si se crean los 100 archivos de 1 GB y todos cargan su contenido:

Escenario	Memoria aproximada del contenido
Sin Proxy: 100 archivos cargados	100 GB
Con Proxy: 0 archivos accedidos	$\sim$0 GB + metadatos
Con Proxy: k archivos accedidos	k GB + metadatos
Con Proxy: 100 archivos accedidos	100 GB + sobrecoste del proxy

El ahorro real es:

$$MemoriasinProxy = N \times T$$

$$MemoriaconProxy = M \times T + overhead$$

donde N es total de archivos, T es tamaño por archivo y M es cantidad de archivos efectivamente cargados. Si solo se usan 10 de 100 archivos de 1 GB, se pasa de $\sim$100 GB a $\sim$10 GB.

Trade-off: el proxy reduce memoria y tiempo de arranque, pero el primer acceso paga la latencia de carga. Los accesos posteriores son rápidos. Si todos los archivos se acaban usando, el proxy no ahorra memoria y añade una indirección mínima. Las métricas permiten decidir si conviene cargar, cachear o descargar.

Ejercicio 6 — Sistema de Control de Tráfico Aéreo

a)

El patrón que mejor encaja es Mediator. Los aviones no deben conocerse ni comunicarse directamente entre sí, ya que eso crearía una red de dependencias caótica; por el contrario, un Mediator centraliza la coordinación de modo que los aviones envían solicitudes a la torre y la torre decide, ordena y responde, mientras cada avión solo conoce la interfaz del mediador.

Clases propuestas:

- `IMediadorTrafico`: interfaz del mediador con métodos `Registrar(Avion)` y `Enviar(Mensaje)`.
- `TorreControl`: mediador concreto que mantiene aviones registrados, colas FIFO por avión y procesa mensajes.
- `Avion`: colega con `Id`, referencia a `IMediadorTrafico` y estado.
- `Mensaje`: contiene el tipo (`Aterrizaje`, `Despegue`, `CambioAltitud`) y los datos asociados.
- Se pueden crear 3 instancias de `TorreControl` y asignar cada avión a una torre.

Por un lado, Observer sirve para notificar cambios, pero no para coordinar decisiones, resolver conflictos ni garantizar orden FIFO por avión; por otro lado, un Pub/Sub directo entre aviones seguiría dejando la lógica de coordinación distribuida y podría generar caos. En cambio, el Mediator centraliza el protocolo, el estado y el ordenamiento.

b)

```
using System;
using System.Collections.Concurrent;
using System.Collections.Generic;
using System.Linq;

public enum TipoMensaje
{
    Aterrizaje,
    Despegue,
    CambioAltitud
}

public sealed class Mensaje
{
    public string AvionId { get; }
    public TipoMensaje Tipo { get; }
    public int Altitud { get; }

    public Mensaje(string avionId, TipoMensaje tipo, int altitud = 0)
    {
        AvionId = avionId;
        Tipo = tipo;
        Altitud = altitud;
    }
}

public interface IMediadorTrafico
{
    void Registrar(Avion avion);
    void Enviar(Mensaje mensaje);
}

public sealed class TorreControl : IMediadorTrafico
{
    public string Nombre { get; }

    private readonly ConcurrentDictionary<string, Avion> _aviones = new();
    private readonly ConcurrentDictionary<string, ConcurrentQueue<Mensaje>> _colas =
    new();
    private readonly ConcurrentDictionary<string, object> _candadosPorAvion = new();
```

```
    public TorreControl(string nombre) => Nombre = nombre;

    public void Registrar(Avion avion)
    {
        _aviones[avion.Id] = avion;
        _colas[avion.Id] = new ConcurrentQueue<Mensaje>();
        _candadosPorAvion[avion.Id] = new object();
    }

    public void Enviar(Mensaje mensaje)
    {
        var candado = _candadosPorAvion.GetOrAdd(mensaje.AvionId, _ => new object());

        lock (candado)
        {
            _colas[mensaje.AvionId].Enqueue(mensaje);
            ProcesarCola(mensaje.AvionId);
        }
    }

    private void ProcesarCola(string avionId)
    {
        while (_colas[avionId].TryDequeue(out var mensaje))
        {
            switch (mensaje.Tipo)
            {
                case TipoMensaje.Aterrizaje:
                    Console.WriteLine($"{Nombre}: autoriza aterrizaje de {avionId}");
                    break;
                case TipoMensaje.Despegue:
                    Console.WriteLine($"{Nombre}: autoriza despegue de {avionId}");
                    break;
                case TipoMensaje.CambioAltitud:
                    Console.WriteLine($"{Nombre}: {avionId} cambia altitud a
    {mensaje.Altitud} pies");
                    break;
            }
        }
    }
}

public sealed class Avion
{
    public string Id { get; }
    private readonly IMediadorTrafico _torre;

    public Avion(string id, IMediadorTrafico torre)
    {
        Id = id;
        _torre = torre;
        _torre.Registrar(this);
    }

    public void SolicitarAterrizaje() => _torre.Enviar(new Mensaje(Id,
    TipoMensaje.Aterrizaje));
    public void SolicitarDespegue() => _torre.Enviar(new Mensaje(Id,
    TipoMensaje.Despegue));
```

```
    public void CambiarAltitud(int pies) => _torre.Enviar(new Mensaje(Id,
    TipoMensaje.CambioAltitud, pies));
}
```

Creación de 3 torres y 10 aviones:

```
var torres = new List<TorreControl>
{
    new TorreControl("Torre Norte"),
    new TorreControl("Torre Sur"),
    new TorreControl("Torre Este")
};

var aviones = Enumerable.Range(1, 10)
    .Select(i => new Avion($"AV{i:00}", torres[(i - 1) % torres.Count]))
    .ToList();

aviones[0].SolicitarAterrizaje();
aviones[1].CambiarAltitud(12000);
aviones[2].SolicitarDespegue();
```

La concurrencia se maneja con un candado por avión. Dos mensajes del mismo avión no se procesan a la vez y mantienen FIFO; mensajes de aviones distintos pueden procesarse concurrentemente. `ConcurrentQueue` evita condiciones de carrera en la cola.

c)

Si una torre falla, los aviones que tiene asignados pierden su mediador y, si además es la única torre, se convierte en un punto único de fallo; para añadir redundancia sin romper el patrón conviene que los aviones dependan de `IMediadorTrafico` y no de `TorreControl`, de modo que se puedan crear torres de respaldo que implementen la misma interfaz. Una estrategia posible es tener una torre primaria y otra secundaria con réplica de colas y estado, heartbeat y reasignación de aviones a la secundaria cuando la primaria no responda, aunque también puede haber tres torres activas por sectores con respaldo entre ellas; así no se rompe el Mediator porque el avión sigue hablando con un mediador abstracto y solo cambia la instancia concreta. Para no perder mensajes, las colas o el estado crítico deberían replicarse o persistirse, ya que, si no hay redundancia, la torre sí es un punto único de fallo.

d)

Command encapsula cada solicitud como objeto:

```
public interface IComandoTrafico
{
    void Ejecutar();
}

public sealed class ComandoAterrizaje : IComandoTrafico
{
    private readonly Avion _avion;
    public ComandoAterrizaje(Avion avion) => _avion = avion;
    public void Ejecutar() => _avion.SolicitarAterrizaje();
}

public sealed class ComandoDespegue : IComandoTrafico
{
    private readonly Avion _avion;
    public ComandoDespegue(Avion avion) => _avion = avion;
```

```
    public void Ejecutar() => _avion.SolicitarDespegue();
}

public sealed class ComandoCambioAltitud : IComandoTrafico
{
    private readonly Avion _avion;
    private readonly int _altitud;
    public ComandoCambioAltitud(Avion avion, int altitud)
    {
        _avion = avion;
        _altitud = altitud;
    }
    public void Ejecutar() => _avion.CambiarAltitud(_altitud);
}
```

State modela estados del avión:

```
public interface IEstadoAvion
{
    void SolicitarAterrizaje(Avion avion);
    void SolicitarDespegue(Avion avion);
    void CambiarAltitud(Avion avion, int altitud);
}

public sealed class EnVuelo : IEstadoAvion
{
    public void SolicitarAterrizaje(Avion avion)
    {
        avion.Enviar(TipoMensaje.Aterrizaje);
        avion.CambiarEstado(new Aterrizando());
    }

    public void SolicitarDespegue(Avion avion) =>
        Console.WriteLine($"{avion.Id} ya est en vuelo");

    public void CambiarAltitud(Avion avion, int altitud) =>
        avion.Enviar(TipoMensaje.CambioAltitud, altitud);
}

public sealed class Aterrizando : IEstadoAvion
{
    public void SolicitarAterrizaje(Avion avion) =>
        Console.WriteLine($"{avion.Id} ya est aterrizando");

    public void SolicitarDespegue(Avion avion) =>
        Console.WriteLine($"{avion.Id} no puede despegar mientras aterriza");

    public void CambiarAltitud(Avion avion, int altitud) =>
        Console.WriteLine($"{avion.Id} no cambia altitud mientras aterriza");
}

public sealed class EnTierra : IEstadoAvion
{
    public void SolicitarAterrizaje(Avion avion) =>
        Console.WriteLine($"{avion.Id} ya est en tierra");

    public void SolicitarDespegue(Avion avion)
    {
        avion.Enviar(TipoMensaje.Despegue);
```

```
        avion.CambiarEstado(new EnVuelo());
    }

    public void CambiarAltitud(Avion avion, int altitud) =>
        Console.WriteLine($"{avion.Id} est en tierra; no aplica cambio de altitud");
}
```

El `Avion` mantiene su estado y delega:

```
public sealed class Avion
{
    public string Id { get; }
    private readonly IMediadorTrafico _torre;
    private IEstadoAvion _estado = new EnVuelo();

    public Avion(string id, IMediadorTrafico torre)
    {
        Id = id;
        _torre = torre;
        _torre.Registrar(this);
    }

    public void CambiarEstado(IEstadoAvion nuevo) => _estado = nuevo;
    public void Enviar(TipoMensaje tipo, int altitud = 0) =>
        _torre.Enviar(new Mensaje(Id, tipo, altitud));

    public void SolicitarAterrizaje() => _estado.SolicitarAterrizaje(this);
    public void SolicitarDespegue() => _estado.SolicitarDespegue(this);
    public void CambiarAltitud(int altitud) => _estado.CambiarAltitud(this, altitud);
}
```

Así, Command encapsula la solicitud, State decide si la solicitud es válida según el estado y Mediator centraliza la coordinación y el orden FIFO por avión.